第二课-函数与面向对象

本课程以 C++17 为语言上限。教学代码使用显式类型，标量采用等号初始化，普通对象采用圆括号构造；数组和聚合数据采用等号后的初始化列表。

这是一份可连续阅读的完整讲义。正文按去年仓库的演示顺序展开；每个实验包括概念推导、完整单文件程序、状态分析、错误定位及带答案的自检。配套题库另有 120 道独立例题，函数与指针练习随原第二课归位。

运行平台为 CPP-Classroom，选择 C++17，整份粘贴一个 .cpp 后运行。平台只运行程序，不内置本课程的自动判题。所有可变输入在题目左侧预填，或运行时按行发送；读到流结束的程序需要发送 EOF。

Original repository: unnc-aim/aim-2526-rookie-courses

章节目录

- 01. 函数：输入、计算与返回
- 02. 引用、地址与指针
- 03. 用交换实验理解参数传递
- 04. 从 struct 到 Band 类
- 05. Band 的构造与初始化
- 06. 实例、方法与乐队集合
- 07. MofoBand：继承与基类构造
- 08. protected、this 与第二层派生
- 09. 原课程整合与工程拓展

01 函数：输入、计算与返回

原稿定位: test_add，保持原第二课起点

1.1 从 test_add 读懂函数接口

第二课沿用原稿的 test_add 开场，而不是先从继承或资源管理讲起。int test_add(int a,int b) 把接口完整写出：它接收两个 int 参数，返回一个 int。调用 test_add(1,2) 时，实参 1 与 2 用来初始化形参 a 与 b，函数体在自己的作用域中运行。

原函数在内部打印一条演示提示，再计算 sum 并 return。提示输出和返回值是两条不同的信息通道：cout 面向屏幕，return 面向调用者。不能通过“函数已经打印出 3”推断调用表达式一定返回 3。把 return 删除或误写成输出，会改变接口行为。

调用者可以把返回值赋给变量，也可以继续参与更大的表达式。函数里的局部 sum 在返回后结束生命周期，但它的整数值已被交给调用者。返回基本数值不要求把局部变量变成全局变量。

完整课堂实验

L2_C01_D01.cpp · CPP-Classroom source

```
#include <iostream>

int test_add(int a, int b) {
    int sum = a + b;
    return sum;
}

int main() {
    int result = test_add(1, 2);
    std::cout << result << '\n';
    return 0;
}
```

执行推导

进入函数后 a=1、b=2、sum=3；return 把 3 交给调用点，result 得到 3，最后输出 3。

常见错误及原因

返回类型为 int 的普通函数应在需要返回结果的路径上提供匹配的值。不要把 main 末尾的特殊规则推广到所有函数。

自检

std::cout << test_add(2,3) 会先得到什么值再输出？

参考解释： 调用先产生整数返回值 5，再由外层输出流输出；函数自己的提示若存在也会执行。

1.2 声明、定义与 void

编译器在调用处需要知道函数的声明。把定义放在 main 前面是入门示例最直接的组织方式；若定义写在后面，就在前面先放声明。声明以分号结束，定义提供大括号内的实现。名字、参数与返回类型必须匹配，不能靠“看起来差不多”完成链接。

void 表示不返回一个结果值，不表示函数什么都不做。原 cite_pointer、swap1、swap2 和 print_info 都通过输出或修改对象产生作用，所以适合讨论副作用。一个函数可以没有参数，也可以既有参数又没有返回值；接口要按职责选择。

CPP-Classroom 的本课程序保持单文件：把声明、定义和 main 放在同一份源码里。原仓库的头文件与 cpp 分离保留为工程组织说明，但不要求学生在尚不支持多文件工程的平台上创建多个编译单元。

完整课堂实验

L2_C01_D02.cpp · CPP-Classroom source

```
#include <iostream>

int test_add(int a, int b) { return a + b; }
void print_sum(int a, int b) { std::cout << test_add(a, b) << '\n'; }

int main() {
    print_sum(4, 5);
    return 0;
}
```

执行推导

print_sum 接收两个整数，在内部调用 test_add，再输出结果。调用者没有接收返回值，因为 print_sum 的返回类型是 void。

常见错误及原因

只有声明而没有对应实现，可能通过语法检查却在链接时失败；这与参数类型根本不匹配的编译错误不同。

自检

void 函数能否写 return; 提前结束?

参考解释： 可以，不携带返回值的 return 结束该次调用。

本章配套例题

ID	题目
S49	返回值与参数
S50	声明与定义
S51	速度限幅函数
S52	默认参数
S53	函数重载
S54	局部作用域
S55	递归阶乘
S56	布尔结果与输出参数

02 引用、地址与指针

原稿定位: cite_pointer：保留 band1/live1/anime1

2.1 引用是已有对象的别名

原 cite_pointer 定义 string band1="poppin party"，然后 string& live1=band1。live1 不是复制出来的第二份字符串，而是访问同一个字符串对象的另一个名字。通过 live1 写入，会改变之后通过 band1 读到的内容。

引用通常在声明时绑定目标，之后不会因为赋值而改绑。写 live1=another 的意思是把 another 的值赋给当前绑定的字符串，不是让 live1 从此指向另一个对象。这个区别会直接影响 swap 的实现和对象参数传递。

原程序打印 &band1 与 &live1，目的是验证两个名字关联同一对象。真实地址因运行而异，不能把某次十六进制地址抄成固定样例答案。课堂用 &band1==&live1 的布尔结果做稳定验证，同时保留讲师可选的地址观察。

完整课堂实验

L2_C02_D01.cpp · CPP-Classroom source

```
#include <iostream>
#include <string>

int main() {
    std::string band1 = "poppin party";
    std::string &live1 = band1;
    live1 = "afterglow";
    std::cout << band1 << '\n';
    std::cout << std::boolalpha << (&band1 == &live1) << '\n';
    return 0;
}
```

执行推导

输出 afterglow 和 true。同一个字符串对象被更新一次，不存在另一份需要同步的 live1 字符串。

常见错误及原因

把 std::string& 改成 std::string 就会创建副本，之后的赋值不再修改 band1。

自检

去掉声明中的 &，输出的 band1 还会变成 afterglow 吗？

参考解释： 不会。live1 成为独立副本，band1 仍为 poppin party。

2.2 指针变量与解引用

原稿先把 `string*` anime1 初始化为 nullptr，再令 anime1=&band1。指针是一个保存地址值的对象；&band1 取得 band1 的地址，赋给 anime1 后，指针知道要关联哪个字符串。注意类型是 `string*`，与字符串本身的 string 不同。

在表达式中 `*anime1` 表示解引用，即访问指针所指对象。std::cout<<anime1 输出地址表示，std::cout<<`*anime1` 输出字符串内容。声明中的 * 与表达式中的 * 位置不同、职责不同；不要只记成“星号就是乘法”。

通过 `*anime1`="afterglow" 修改的是目标字符串，并不改变指针中保存的地址。反过来，anime1=&band2 改变的是指针所指目标，不会自动改写 band1。用“指针盒子”和“目标对象”两个独立状态来追踪，比混称“改了指针”更准确。

完整课堂实验

`L2_C02_D02.cpp` · CPP-Classroom source

```
#include <iostream>
#include <string>

int main() {
    std::string band1 = "poppin party";
    std::string band2 = "mygo";
    std::string *anime1 = &band1;
    *anime1 = "afterglow";
    anime1 = &band2;
    std::cout << band1 << ' ' << *anime1 << '\n';
    return 0;
}
```

执行推导

第一次写入把 band1 改为 afterglow；随后指针改指 band2。最终输出 afterglow mygo，band1 不会被后一次重定向改回去。

常见错误及原因

指针变量本身也需要初始化。未初始化指针不等于 nullptr，更不能直接解引用。

自检

p=&x 与 *p=x 的修改对象分别是谁?

参考解释： 前者修改指针 p 保存的地址，后者修改 p 当前指向的对象。

2.3 空指针、只读借用与有效期

nullptr 表示没有指向对象，不能通过它访问某个字符串。原例在赋予有效地址后才解引用，修订版把这个前提明确写出来。需要允许空值的接口，应先检查指针；要求对象必定存在的接口，可以考虑引用，但引用也需要目标在使用期间始终存活。

const string& 允许读取而不复制整个字符串，也不允许通过这个引用修改它。const `string*` 限制通过指针修改目标，但指针可以改指别处；`string* const` 则固定指针自身，目标仍可能可修改。先分清 const 修饰哪一层，再讨论接口是否只读。

函数返回后，其普通局部对象会销毁。返回局部变量地址或引用会留下悬空访问入口。要返回计算结果或新建字符串，通常直接按值返回即可。不要为了“避免复制”而返回一个已经不存在的对象。

完整课堂实验

`L2_C02_D03.cpp` · CPP-Classroom source

```
#include <iostream>
#include <string>

int main() {
    std::string band = "mygo";
    const std::string *p = nullptr;
    if (p != nullptr) {
        std::cout << *p << '\n';
    }
    p = &band;
    std::cout << *p << '\n';
    return 0;
}
```

执行推导

第一次检查为空，所以不执行解引用；给 p 赋地址后输出 mygo。指针只借用 band，示例没有动态分配，也不应 delete 这个地址。

常见错误及原因

“非空”只排除空值，不证明目标仍存活。悬空指针也可能保存非零地址。

自检

返回局部 string 的 const 引用，会因为 const 而延长其生命周期吗？

参考解释： 不会。const 不会挽救已在函数结束时销毁的局部对象。

本章配套例题

ID	题目
S59	指针读写
S60	空指针保护
S61	指向 const 的指针
S62	const 引用遍历
S63	按值返回避免悬空
S64	非拥有指针区间

03 用交换实验理解参数传递

原稿定位: swap1；swap2；引用版本作为对照补充

3.1 swap1 为什么没有交换调用者

原 swap1(int a,int b) 在函数内部使用 temp 完成三次赋值。算法本身确实交换了 a 与 b，但这两个形参是调用时从实参复制来的整数对象。调用者 x、y 与函数内部 a、b 是不同对象，因此内部交换没有修改外部变量。

必须沿着一次调用画出四个变量：外部 x=10、y=20，进入函数后 a=10、b=20。temp 保存 10，a 变成 20，b 变成 10；函数返回时 a、b、temp 结束生命周期，外部仍是 10 与 20。只在函数里打印参数会误以为交换任务已经完成。

这不是编译器缺陷，也不是 temp 的写法错误，而是接口选择的结果。按值传递适合只需要输入副本的计算；若函数必须修改调用者，接口要显式借用原对象，使用引用或指向原对象的指针。

完整课堂实验

L2_C03_D01.cpp · CPP-Classroom source

```
#include <iostream>

void swap1(int a, int b) {
    int temp = a;
    a = b;
    b = temp;
}

int main() {
    int x = 10, y = 20;
    swap1(x, y);
    std::cout << x << ' ' << y << '\n';
    return 0;
}
```

执行推导

输出 10 20。函数内部最终 a=20、b=10，不影响这个结果。

Step	State / operation	Result
call	x=10,y=20	a=10,b=20
temp=a	x=10,y=20	temp=10
a=b	x=10,y=20	a=20
b=temp	x=10,y=20	b=10
return	x=10,y=20	parameters destroyed

常见错误及原因

不要把“函数体里值变了”当成“实参已经变了”。两处输出应分别验证。

自检

如果把 swap1 的返回类型改成 int，但仍不把结果赋回 x、y，会自动交换吗？

参考解释： 不会。返回类型不会改变形参的按值语义。

3.2 swap2 通过地址修改原对象

原 swap2 接收 `int* p1`、`int* p2`，调用时传 &x、&y。形参指针本身仍按值复制，但复制的是地址。函数内的 p1 与调用者传入的地址值关联同一个 x，所以 *p1 的写入会落在 x 上。说“指针参数不是按值传递”并不准确。

三步交换现在作用于目标：temp=*p1 保存原 x，*p1=*p2 把原 y 写入 x，*p2=temp 再写入 y。最后两个外部变量真实改变。前提是两个指针都指向有效、可写的 int；若接口允许空指针，必须先定义并实现空值行为。

引用版本 swap_ref(int& a,int& b) 可以用直接变量语法修改原对象，调用时不用写取地址。两者的选择取决于是否允许“无对象”、是否需要重定向指针以及接口约定。用哪种写法都要能说明被修改的到底是哪一个对象。

完整课堂实验

`L2_C03_D02.cpp` · CPP-Classroom source

```
#include <iostream>

void swap2(int *p1, int *p2) {
    int temp = *p1;
    *p1 = *p2;
    *p2 = temp;
}
void swap_ref(int &a, int &b) {
    int temp = a;
    a = b;
    b = temp;
}

int main() {
    int x = 10, y = 20;
    swap2(&x, &y);
    std::cout << x << ' ' << y << '\n';
    swap_ref(x, y);
    std::cout << x << ' ' << y << '\n';
    return 0;
}
```

执行推导

swap2 后输出 20 10；引用版本再交换一次，输出 10 20。两次调用都修改同一组外部对象。

Step	State / operation	Result
`call`	`p1=&x,p2=&y`	`x=10,y=20`
`temp=*p1`	`temp=10`	`x=10,y=20`
`*p1=*p2`	`write x`	`x=20,y=20`
`*p2=temp`	`write y`	`x=20,y=10`

常见错误及原因

不要交换 p1、p2 本身来代替交换 *p1、*p2。只交换局部地址副本不会交换 x、y。

自检

swap2(&x,&x) 会破坏 x 吗?

参考解释： 此处三步算法不会；两个地址相同，最终仍写回原值。仍需要 x 本身有效。

本章配套例题

ID	题目
S57	值传递不能交换实参
S58	引用交换

04 从 struct 到 Band 类

原稿定位: Band：保留乐队成员字段，不改为机器人主线

4.1 数据记录为什么需要行为边界

原 Band 把吉他、主唱、鼓、贝斯、键盘五个字段放进同一个类型。先把它视为第一课 struct 的自然延续：每一个 Band 对象代表一支乐队，每个字段代表该乐队的一项状态。同一种类型可以创建多支乐队，而不是让整个程序只剩一套全局成员名字。

如果字段公开，任何外部代码都能随意写入，类型本身无法保证成员修改遵循规则。private 把直接访问限制在类内部，再由 public 成员函数决定允许怎样读取或修改。封装的重点是可执行的状态约束，而不是机械地给每个字段写同名 getter 和 setter。

struct 和 class 都能有成员函数、构造函数和继承。语言层面主要区别是默认成员访问与默认继承权限：struct 默认为 public，class 默认为 private。原教程从 struct 改成 class 的动作便于教学，但不能据此说 struct 没有面向对象能力。

完整课堂实验

`L2_C04_D01.cpp` · CPP-Classroom source

```
#include <iostream>
#include <string>

struct BandRecord {
    std::string guitar, vocal, bass;
};

int main() {
    BandRecord band = {"mutsumi", "hatsune", "umiri"};
    std::cout << band.guitar << ':' << band.vocal << '\n';
    return 0;
}
```

执行推导

这个过渡示例仍允许外部直接读写字段。下一节把同样的状态放到 private 中，访问方式随之改变。

常见错误及原因

“用了 class 关键字”不自动等于已经完成良好封装；全部字段公开时仍缺少修改边界。

自检

把 struct 改为 class 而不补 public，原来的外部字段访问为什么失败?

参考解释： 成员默认访问权限从 public 变成 private，调用者不再能直接访问。

4.2 private、public 与成员函数调用

读 Band 的类定义时，先找访问标签，再读每个成员属于哪一段。private 下的五个 string 是内部状态，public 下的构造函数和方法是外部可用接口。外部写 band.guitar 会被编译器拒绝；band.get_guitar() 则调用一个有权读取内部状态的成员函数。

点前面的对象决定操作哪支乐队。a.set_keyboard_anon() 修改 a，b 不会跟着变。成员函数内部可以不写对象前缀就访问 guitar，因为调用已经确定了当前对象。稍后用 this 把这个隐含关联写得更明确。

旧注释说构造函数“必须 public”，这是不准确的绝对表述。本课为了让 main 直接创建 Band，选择 public 构造函数；其他设计可用 private 构造配合工厂接口。讲义保留当前示例的设计意图，同时把适用范围说清。

完整课堂实验

L2_C04_D02.cpp · CPP-Classroom source

```
#include <iostream>
#include <string>
#include <utility>

class Band {
  private:
    std::string guitar, vocal, drum, bass, keyboard;

  public:
    Band(std::string gt, std::string vo, std::string dr, std::string ba,
         std::string key) {
        guitar = std::move(gt);
        vocal = std::move(vo);
        drum = std::move(dr);
        bass = std::move(ba);
        keyboard = std::move(key);
    }
    void print_info() const {
        std::cout << guitar << ',' << vocal << ',' << drum << ',' << bass << ','
                  << keyboard << '\n';
    }
    void live() const { std::cout << "LIVE\n"; }
    void get_guitar() const { std::cout << guitar << '\n'; }
    void set_keyboard_anon() { keyboard = "anon"; }
};

int main() {
    Band a("mutsumi", "hatsune", "nyamu", "umiri", "saki");
    Band b("hina", "aya", "maya", "chisato", "eve");
    a.set_keyboard_anon();
    a.print_info();
    b.print_info();
    return 0;
}
```

执行推导

a 的最后字段变为 anon，b 的最后字段仍是 eve。用两个对象一起观察，能明确类定义共享、实例状态独立的区别。

Step	State / operation	Result
`construct a`	`a.keyboard`	`saki`
`construct b`	`b.keyboard`	`eve`
`a.set_keyboard_anon()`	`a.keyboard`	`anon`
`observe b`	`b.keyboard`	`eve`

常见错误及原因

直接访问私有成员的错误应作为单独的故障实验，不要把不能编译的语句留在正常示例里。

自检

get_guitar 能读取 private 字段，是否说明 private 没起作用？

参考解释： 不是。private 限制外部直接访问，类自己的成员函数正是被允许管理这些字段的地方。

本章配套例题

ID	题目
O01	第一个类
O02	两个对象的状态
O03	private 访问边界
O04	拒绝无效修改
O05	struct 与 class

05 Band 的构造与初始化

原稿定位: Band 构造函数；std::move；MofoBand 初始化列表铺垫

5.1 构造参数怎样进入对象状态

创建 Band 时需要五个字符串，分别对应 guitar、vocal、drum、bass、keyboard。构造函数与类同名，没有返回类型。它在对象建立过程中执行，负责让这个对象从一开始就有可使用的状态。普通成员方法是在对象已经存在之后才被调用，两者的时间点不同。

原构造函数先默认构造五个 string 成员，再在函数体里逐个赋值。这对 string 可以工作，但不等于所有成员类型都能如此处理。const 成员、引用成员以及没有默认构造函数的成员，需要在初始化阶段给出合适的初始化信息。下一步才引出成员初始化列表。

原代码用 std::move(gt) 等表达式允许把形参的资源转移给成员。std::move 本身是转换，不是一个“立即搬运所有字节”的函数；是否发生移动由后续操作和类型决定。入门时先把它理解为：这个局部形参之后不再需要原内容，可交给成员接管。

完整课堂实验

L2_C05_D01.cpp · CPP-Classroom source

```
#include <iostream>
#include <string>
#include <utility>

class Band {
  private:
    std::string guitar, vocal, drum, bass, keyboard;

  public:
    Band(std::string gt, std::string vo, std::string dr, std::string ba,
         std::string key) {
        guitar = std::move(gt);
        vocal = std::move(vo);
        drum = std::move(dr);
        bass = std::move(ba);
        keyboard = std::move(key);
    }
    void print_info() const {
        std::cout << guitar << ',' << vocal << ',' << drum << ',' << bass << ','
                  << keyboard << '\n';
    }
    void live() const { std::cout << "LIVE\n"; }
    void get_guitar() const { std::cout << guitar << '\n'; }
    void set_keyboard_anon() { keyboard = "anon"; }
};

int main() {
    Band band("mutsumi", "hatsune", "nyamu", "umiri", "saki");
    band.print_info();
    return 0;
}
```

执行推导

五个实参先初始化形参，再被赋给对应成员。打印顺序由 print_info 的实现决定，而不是由变量声明的排列自动产生。

常见错误及原因

不要在构造函数名前写 void；那会变成一个与类同名但语法不合法的普通函数声明。

自检

创建 Band 时只给四个参数会怎样?

参考解释： 当前接口没有匹配的四参数构造函数，编译失败。

5.2 成员初始化列表与实际顺序

把原构造改写为冒号后的初始化列表，可以直接构造成员。Band(...):guitar(move(gt)),... 的语法放在函数体大括号之前。本课程使用圆括号提供成员的初值，不是一个在运行中随意调用成员变量的函数。

成员按类中声明的顺序初始化，不按初始化列表的书写顺序。若后一个成员依赖前一个成员，声明顺序就有实际含义。基类部分又先于派生类自己的成员建立，这为后面 MofoBand(...):Band(...) 铺好理解路径。

不要为了炫技一次引入委托构造、移动构造与整套特殊成员函数。这里首先解决“对象建立前必须准备什么”的具体问题。额外的对象复制、移动和资源约束在末尾拓展中解释，不替代原稿构造到继承的主线。

完整课堂实验

L2_C05_D02.cpp · CPP-Classroom source

```
#include <iostream>
#include <string>
#include <utility>

class BandInit {
    std::string guitar;
    std::string keyboard;

  public:
    BandInit(std::string gt, std::string key)
        : guitar(std::move(gt)), keyboard(std::move(key)) {}
    void print() const { std::cout << guitar << ':' << keyboard << '\n'; }
};

int main() {
    BandInit band("mutsumi", "saki");
    band.print();
    return 0;
}
```

执行推导

guitar 与 keyboard 在进入空函数体之前就已完成初始化。打印得到 mutsumi:saki。

常见错误及原因

把初始化列表写到构造函数体里面是不合法的；冒号必须跟在参数列表之后、函数体之前。

自检

初始化列表里把 keyboard 写在 guitar 前面，会改变实际成员建立顺序吗？

参考解释： 不会，仍由成员声明顺序决定，应尽量按声明顺序书写以免误导。

本章配套例题

ID	题目
O06	成员初始化列表
O07	const 数据成员
O08	委托构造函数
O09	explicit 防止隐式转换
O10	无默认构造的成员

06 实例、方法与乐队集合

原稿定位: class_band：创建实例、print_info/live、vector、getter/setter，保持原执行次序

6.1 读方法、写方法与 const

原 class_band 先创建 ave_mujica，再调用 print_info 与 live，随后调用 get_guitar 与 set_keyboard_anon。读取与修改的区别由方法行为决定。print_info 只是观察状态，set_keyboard_anon 则改变对象，调用顺序直接决定下一次输出。

本版把不修改状态的方法标为 const。写在成员函数参数列表后面的 const，承诺这次调用不通过当前对象修改普通成员，因此 const Band 也可以调用它。这个 const 不修饰返回值，也不是把整个类所有对象都冻结。

原 get_guitar 是一个输出方法，并不返回 string。保持原接口时，调用它的目的是在屏幕显示吉他手；若工程代码需要继续处理姓名，可另设计返回值接口。不要把 “方法名含 get” 当成语言保证的返回行为。

完整课堂实验

L2_C06_D01.cpp · CPP-Classroom source

```
#include <iostream>
#include <string>
#include <utility>

class Band {
  private:
    std::string guitar, vocal, drum, bass, keyboard;

  public:
    Band(std::string gt, std::string vo, std::string dr, std::string ba,
         std::string key) {
        guitar = std::move(gt);
        vocal = std::move(vo);
        drum = std::move(dr);
        bass = std::move(ba);
        keyboard = std::move(key);
    }
    void print_info() const {
        std::cout << guitar << ',' << vocal << ',' << drum << ',' << bass << ','
                  << keyboard << '\n';
    }
    void live() const { std::cout << "LIVE\n"; }
    void get_guitar() const { std::cout << guitar << '\n'; }
    void set_keyboard_anon() { keyboard = "anon"; }
};

int main() {
    Band ave_mujica("mutsumi", "hatsune", "nyamu", "umiri", "saki");
    ave_mujica.print_info();
    ave_mujica.live();
    ave_mujica.get_guitar();
    ave_mujica.set_keyboard_anon();
    ave_mujica.print_info();
    return 0;
}
```

执行推导

第一次名单最后是 saki，随后输出 LIVE 和 mutsumi，修改键盘手后最后一行的末尾才变成 anon。

常见错误及原因

不能把 void get_guitar() 的调用结果赋给 string；它没有提供这样的返回值。

自检

把 ave_mujica 定义成 const，哪一个调用会被拒绝？

参考解释： set_keyboard_anon，因为它会修改普通成员；只读 const 方法仍可调用。

6.2 `vector<Band>`、emplace_back 与范围引用

原 live_bands 是 `vector<Band>`，容器元素从整数换成了我们定义的对象。emplace_back 把构造所需的五个名字传进去，在容器中建立 Band。它不是把字符串拼成一个名单文本，而是保存多个带行为的对象。

原 for(Band& p:live_bands) 让 p 逐个引用容器中的对象，避免每轮复制整支乐队。若只是打印，本版可以使用 const Band&，既表达只读，也允许在 const 容器上工作。若确实要调用修改方法，就保留非 const 引用。

使用 for(Band p:live_bands) 则每轮复制一个对象。在副本上 set_keyboard_anon 不会改变容器内的原乐队。这个现象与第一课 int value 的数组遍历、第二课 swap1 的按值形参是同一个值复制主题，而不是一个新的神秘规则。

完整课堂实验

`L2_C06_D02.cpp` · CPP-Classroom source

```
#include <iostream>
#include <string>
#include <utility>
#include <vector>

class Band {
  private:
    std::string guitar, vocal, drum, bass, keyboard;

  public:
    Band(std::string gt, std::string vo, std::string dr, std::string ba,
         std::string key) {
        guitar = std::move(gt);
        vocal = std::move(vo);
        drum = std::move(dr);
        bass = std::move(ba);
        keyboard = std::move(key);
    }
    void print_info() const {
        std::cout << guitar << ',' << vocal << ',' << drum << ',' << bass << ','
                  << keyboard << '\n';
    }
    void live() const { std::cout << "LIVE\n"; }
    void get_guitar() const { std::cout << guitar << '\n'; }
```

```
    void set_keyboard_anon() { keyboard = "anon"; }
};

int main() {
    std::vector<Band> live_bands;
    live_bands.emplace_back("hina", "aya", "maya", "chisato", "eve");
    live_bands.emplace_back("sayo", "yukina", "ako", "lisa", "rinko");
    for (const Band &p : live_bands) {
        p.print_info();
    }
    return 0;
}
```

执行推导

容器含两个 Band，按插入顺序输出两行。每次迭代 p 关联当前元素，循环本身不会再制造一个可独立修改的乐队副本。

常见错误及原因

容器增长可能搬迁元素，不要长期保留插入前取得的元素地址。引用遍历期间也不要随意 push_back。

自检

把 const Band& 改成 Band，打印结果一定变化吗？

参考解释： 只打印时可能看不出区别，但会发生复制；修改副本时差异才在状态上显现。

本章配套例题

ID	题目
O11	const 成员函数
O12	this 与同名参数
O13	链式调用返回引用
O14	引用访问与值访问
O15	静态成员函数
O16	对象值复制
O17	引用传参与复制传参
O18	作用域销毁顺序
O19	vector 保存对象
O20	静态活对象计数

07 MofoBand：继承与基类构造

原稿定位: MofoBand : public Band；print_mofo_info

7.1 public 继承建立什么关系

原 MofoBand 在 Band 的五个成员信息之外增加 vocal_actor。public Band 表达这个派生类型可以作为一种 Band 来使用，已有的 public 方法成为可调用接口。派生对象包含一个 Band 基类部分，不是把 Band 的源码简单粘贴了一遍。

Band 的 private 字段仍然由 Band 自己管理。MofoBand 可以调用 print_info 来显示基类状态，却不能直接写 guitar。继承不等于绕过封装，也不是让子类“自动获得任意私有访问权限”。需要的行为应通过基类提供的接口表达。

选择继承前要确认关系。乐队的一种扩展类型可以使用这个教学模型；电机是机器人拥有的部件，通常适合成员组合，而不是 Robot 继承 Motor。组合与继承是不同的组织关系，后面的机器人题库以明确标签作为迁移练习。

完整课堂实验

L2_C07_D01.cpp · CPP-Classroom source

```
#include <iostream>
#include <string>
#include <utility>

class Band {
  private:
    std::string guitar, vocal, drum, bass, keyboard;

  public:
    Band(std::string gt, std::string vo, std::string dr, std::string ba,
         std::string key) {
        guitar = std::move(gt);
        vocal = std::move(vo);
        drum = std::move(dr);
        bass = std::move(ba);
        keyboard = std::move(key);
    }
    void print_info() const {
        std::cout << guitar << ',' << vocal << ',' << drum << ',' << bass << ','
                  << keyboard << '\n';
    }
    void live() const { std::cout << "LIVE\n"; }
    void get_guitar() const { std::cout << guitar << '\n'; }
    void set_keyboard_anon() { keyboard = "anon"; }
};
class MofoBand : public Band {
  protected:
    std::string vocal_actor;

  public:
    MofoBand(std::string gt, std::string vo, std::string dr, std::string ba,
             std::string key, std::string actor)
        : Band(std::move(gt), std::move(vo), std::move(dr), std::move(ba),
               std::move(key)),
```

```
          vocal_actor(std::move(actor)) {}
    void print_mofo_info() const {
        print_info();
        std::cout << vocal_actor << '\n';
    }
};

int main() {
    MofoBand morfonica("toko", "mashiro", "tsukushi", "nanami", "rui", "amane");
    morfonica.print_info();
    morfonica.print_mofo_info();
    return 0;
}
```

执行推导

第一条调用打印基类的五个字段；第二条先打印同样五个字段，再额外输出 amane。基类状态只存一份。

常见错误及原因

调用继承来的 public 方法与直接访问基类 private 字段不是一回事。

自检

MofoBand 的成员函数能否直接写 guitar="new"?

参考解释： 不能，guitar 是 Band 的 private 成员，应通过 Band 允许的接口管理。

7.2 为什么冒号后要调用 Band

原 MofoBand 的构造函数接收六个字符串，前五个用于初始化基类，最后一个用于派生类新增成员。冒号后的 Band(...) 不是普通函数调用语句，而是指定怎样建立基类子对象。基类先完成构造，派生类才继续建立自己的成员并执行构造函数体。

Band 只有需要五个参数的构造函数，没有可用的无参数版本。因此这个 MofoBand 不能把基类初始化留空，再指望在函数体里补做。旧注释说“所有子类都必须自己定义构造函数”过于绝对；准确说法是这个设计必须为不能默认构造的基类提供初始化参数。

本版把 vocal_actor 也放入成员初始化列表，保持“先完整建立对象，再执行方法”的规则一致。初始化的实际次序是基类 Band、派生成员 vocal_actor、派生构造函数体，与方法在类定义里的排列位置无关。

完整课堂实验

L2_C07_D02.cpp · CPP-Classroom source

```
#include <iostream>
#include <string>
#include <utility>

class Band {
  private:
    std::string guitar, vocal, drum, bass, keyboard;

  public:
    Band(std::string gt, std::string vo, std::string dr, std::string ba,
```

```
         std::string key) {
        guitar = std::move(gt);
        vocal = std::move(vo);
        drum = std::move(dr);
        bass = std::move(ba);
        keyboard = std::move(key);
    }
    void print_info() const {
        std::cout << guitar << ',' << vocal << ',' << drum << ',' << bass << ','
                  << keyboard << '\n';
    }
    void live() const { std::cout << "LIVE\n"; }
    void get_guitar() const { std::cout << guitar << '\n'; }
    void set_keyboard_anon() { keyboard = "anon"; }
};
class MofoBand : public Band {
  protected:
    std::string vocal_actor;

  public:
    MofoBand(std::string gt, std::string vo, std::string dr, std::string ba,
             std::string key, std::string actor)
        : Band(std::move(gt), std::move(vo), std::move(dr), std::move(ba),
               std::move(key)),
          vocal_actor(std::move(actor)) {}
    void print_mofo_info() const {
        print_info();
        std::cout << vocal_actor << '\n';
    }
};

int main() {
    MofoBand band("toko", "mashiro", "tsukushi", "nanami", "rui", "amane");
    band.print_mofo_info();
    return 0;
}
```

执行推导

先建立五字段 Band，再建立声优成员，随后才可以安全调用显示完整信息的方法。

Step	State / operation	Result
1	`Band base`	`five member strings`
2	`MofoBand member`	`vocal_actor`
3	`constructor body`	`complete derived object`

常见错误及原因

在函数体里写 Band(...); 只会构造一个临时对象，不能初始化当前对象中早已需要建立的基类部分。

自检

如果删掉 :Band(...)，编译器会尝试寻找什么？

参考解释： 它需要默认初始化 Band；当前 Band 没有匹配的无参数构造函数，所以编译失败。

本章配套例题

ID	题目
O26	public 继承
O27	基类构造委托
O29	构造与析构的层次
O30	同名隐藏与限定调用

08 protected、this 与第二层派生

原稿定位: MofoBand_New；change_actor

8.1 protected 的准确访问范围

原 MofoBand 把 vocal_actor 放在 protected 下，MofoBand_New::change_actor 因而能访问它。这里必须修正原注释“当前类不能访问”：声明该成员的当前类当然能访问，符合 C++ 规则的派生类成员函数也能访问；普通外部代码不能直接访问。

public、protected、private 是编译器检查的访问规则，不是内存加密机制。给字段加 protected 会让派生类依赖基类内部表示，因此工程里也应考虑是否能用更窄的 protected 方法或 public 接口保持约束。这里先忠实解释原稿的继承链。

两层继承都保留 public：MofoBand_New 是 MofoBand 的一种扩展，同时仍包含 Band 基类部分。创建它时仍需把五个成员名和声优参数向上传递。不能只添加 change_actor，就忘掉父类构造所需的数据。

完整课堂实验

L2_C08_D01.cpp · CPP-Classroom source

```
#include <iostream>
#include <string>
#include <utility>

class Band {
  private:
    std::string guitar, vocal, drum, bass, keyboard;

  public:
    Band(std::string gt, std::string vo, std::string dr, std::string ba,
         std::string key) {
        guitar = std::move(gt);
        vocal = std::move(vo);
        drum = std::move(dr);
        bass = std::move(ba);
        keyboard = std::move(key);
    }
    void print_info() const {
        std::cout << guitar << ',' << vocal << ',' << drum << ',' << bass << ','
                  << keyboard << '\n';
    }
    void live() const { std::cout << "LIVE\n"; }
    void get_guitar() const { std::cout << guitar << '\n'; }
    void set_keyboard_anon() { keyboard = "anon"; }
};
class MofoBand : public Band {
  protected:
    std::string vocal_actor;

  public:
    MofoBand(std::string gt, std::string vo, std::string dr, std::string ba,
             std::string key, std::string actor)
        : Band(std::move(gt), std::move(vo), std::move(dr), std::move(ba),
               std::move(key)),
          vocal_actor(std::move(actor)) {}
```

```
    void print_mofo_info() const {
        print_info();
        std::cout << vocal_actor << '\n';
    }
};
class MofoBand_New : public MofoBand {
  public:
    MofoBand_New(std::string gt, std::string vo, std::string dr, std::string ba,
                 std::string key, std::string actor)
        : MofoBand(std::move(gt), std::move(vo), std::move(dr), std::move(ba),
                   std::move(key), std::move(actor)) {}
    void change_actor(const std::string &new_actor) {
        this->vocal_actor = new_actor;
    }
};

int main() {
    MofoBand_New band("toko", "mashiro", "tsukushi", "nanami", "rui", "amane");
    band.change_actor("ayane");
    band.print_mofo_info();
    return 0;
}
```

执行推导

构造时声优为 amane，change_actor 通过派生类方法把它改为 ayane，随后显示新值。

常见错误及原因

main 不能直接写 band.vocal_actor；应调用公开的 change_actor。

自检

MofoBand 自己的 print_mofo_info 能否读取 protected vocal_actor?

参考解释： 可以，protected 不会禁止声明它的类访问自身成员。

8.2 this 指向当前对象，const& 借用参数

在 change_actor 中，this 指向当前这一次调用所操作的 MofoBand_New 对象。this->vocal_actor 明确表示“这个对象的成员”，与直接写 vocal_actor 在这里效果相同。如果形参与成员同名，this 可以帮助区分，但使用不同的命名风格也能减少遮蔽。

const string& new_actor 借用调用方字符串作为只读输入，避免先按值复制一份参数。把它赋给 vocal_actor 时，成员取得自己的字符串值，而不是把一个短命参数引用永久保存下来。因此方法返回后，成员仍可以安全使用。

this 不是 static 成员函数拥有的隐含参数，因为 static 方法不绑定某个具体对象。也不要把 this 与返回当前对象的 `*this` 混淆：一个是指针，一个是所指对象。链式调用等写法可以在拓展题里进一步体验。

完整课堂实验

L2_C08_D02.cpp · CPP-Classroom source

```
#include <iostream>
```

```cpp
#include <string>
#include <utility>

class Band {
  private:
    std::string guitar, vocal, drum, bass, keyboard;

  public:
    Band(std::string gt, std::string vo, std::string dr, std::string ba,
         std::string key) {
        guitar = std::move(gt);
        vocal = std::move(vo);
        drum = std::move(dr);
        bass = std::move(ba);
        keyboard = std::move(key);
    }
    void print_info() const {
        std::cout << guitar << ',' << vocal << ',' << drum << ',' << bass << ','
                  << keyboard << '\n';
    }
    void live() const { std::cout << "LIVE\n"; }
    void get_guitar() const { std::cout << guitar << '\n'; }
    void set_keyboard_anon() { keyboard = "anon"; }
};
class MofoBand : public Band {
  protected:
    std::string vocal_actor;

  public:
    MofoBand(std::string gt, std::string vo, std::string dr, std::string ba,
             std::string key, std::string actor)
        : Band(std::move(gt), std::move(vo), std::move(dr), std::move(ba),
               std::move(key)),
          vocal_actor(std::move(actor)) {}
    void print_mofo_info() const {
        print_info();
        std::cout << vocal_actor << '\n';
    }
};
class MofoBand_New : public MofoBand {
  public:
    MofoBand_New(std::string gt, std::string vo, std::string dr, std::string ba,
                 std::string key, std::string actor)
        : MofoBand(std::move(gt), std::move(vo), std::move(dr), std::move(ba),
                   std::move(key), std::move(actor)) {}
    void change_actor(const std::string &new_actor) {
        this->vocal_actor = new_actor;
    }
};

int main() {
    MofoBand_New a("toko", "mashiro", "tsukushi", "nanami", "rui", "amane");
    {
        std::string replacement = "ayane";
        a.change_actor(replacement);
    }
    a.print_mofo_info();
    return 0;
}
```

执行推导

replacement 的生命周期结束后，a 的 vocal_actor 仍保存 ayane，因为成员接收了字符串值。

常见错误及原因

如果把成员改成引用并绑定一个短期参数，生命周期要求会完全不同，不能照搬当前例子的安全结论。

自检

调用 a.change_actor 后，另一个独立对象 b 的声优会改变吗？

参考解释： 不会，this 只指向接收这次调用的 a。

本章配套例题

ID	题目
O28	protected 的可见性

09 原课程整合与工程拓展

原稿定位: class_band 整合之后才进入扩展；扩展不替换原主线

9.1 把 class_band 作为一条状态轨迹阅读

原 main 最后只开启 class_band，其余演示通过注释逐段切换。课堂版把每个实验做成独立单文件，以免学生忘记取消哪一行注释。整合程序仍保留原顺序：创建基础乐队、显示和演出、建立集合并遍历、读取和修改字段、创建派生乐队、通过第二层派生修改声优。

阅读时同时记录“对象类型”“变量名”“当前字段值”“可调用接口”四列。类型决定拥有哪些接口，变量名标识哪个实例，状态随方法调用改变，访问权限决定哪些表达式能通过编译。这比把整段程序按行翻译成中文更接近实际调试方法。

完成本节后，学员应能口头解释 test_add、live1、anime1、swap1、swap2、Band、MofoBand 和 MofoBand_New 的作用，并能修改参数后预判输出。若不能说明这些原稿例子的行为，不应靠继续堆砌虚函数或智能指针术语来掩盖基础理解问题。

完整课堂实验

L2_C09_D01.cpp · CPP-Classroom source

```
#include <iostream>
#include <string>
#include <utility>

class Band {
  private:
    std::string guitar, vocal, drum, bass, keyboard;

  public:
    Band(std::string gt, std::string vo, std::string dr, std::string ba,
         std::string key) {
        guitar = std::move(gt);
        vocal = std::move(vo);
        drum = std::move(dr);
        bass = std::move(ba);
        keyboard = std::move(key);
    }
    void print_info() const {
        std::cout << guitar << ',' << vocal << ',' << drum << ',' << bass << ','
                  << keyboard << '\n';
    }
    void live() const { std::cout << "LIVE\n"; }
    void get_guitar() const { std::cout << guitar << '\n'; }
    void set_keyboard_anon() { keyboard = "anon"; }
};
class MofoBand : public Band {
  protected:
    std::string vocal_actor;

  public:
    MofoBand(std::string gt, std::string vo, std::string dr, std::string ba,
             std::string key, std::string actor)
        : Band(std::move(gt), std::move(vo), std::move(dr), std::move(ba),
               std::move(key)),
```

```
          vocal_actor(std::move(actor)) {}
    void print_mofo_info() const {
        print_info();
        std::cout << vocal_actor << '\n';
    }
};
class MofoBand_New : public MofoBand {
  public:
    MofoBand_New(std::string gt, std::string vo, std::string dr, std::string ba,
                 std::string key, std::string actor)
        : MofoBand(std::move(gt), std::move(vo), std::move(dr), std::move(ba),
                   std::move(key), std::move(actor)) {}
    void change_actor(const std::string &new_actor) {
        this->vocal_actor = new_actor;
    }
};

int main() {
    Band basic("mutsumi", "hatsune", "nyamu", "umiri", "saki");
    basic.print_info();
    basic.set_keyboard_anon();
    basic.print_info();
    MofoBand_New derived("toko", "mashiro", "tsukushi", "nanami", "rui", "amane");
    derived.change_actor("ayane");
    derived.print_mofo_info();
    return 0;
}
```

执行推导

依次观察基础对象修改前后两行，再观察派生对象的五字段行和声优行。不同对象的状态不应相互串改。

常见错误及原因

只看最终输出不足以定位中途哪个动作写错；应在关键状态转换后建立检查点。

自检

为什么这里没有要求基类出现 virtual?

参考解释： 原课程使用的是继承来的接口与新增方法，没有通过基类引用要求动态派发；继承和运行时多态需要区分。

9.2 拓展：继承、隐藏与虚函数重写

原稿到 protected 和 this 为止，没有系统展开 virtual。本节明确标为拓展：当调用方只知道基类接口，却需要派生对象选择具体行为时，才需要虚函数动态派发。相同名字本身不会自动形成运行时多态。

基类函数声明 virtual，派生函数写 override，编译器就能核对重写签名；末尾 const 也必须匹配。通过基类引用或指针调用时，实际对象的动态类型决定虚函数实现。把派生对象复制成基类值会丢失派生部分，称为对象切片。

若通过基类指针销毁派生对象，基类析构通常必须为 virtual。这个要求与原 Band 的按值对象不同，不能无背景地把所有类都改成虚类。先确认生命周期和调用路径，再选择类型设计。

完整课堂实验

L2_C09_D02.cpp · CPP-Classroom source

```
#include <iostream>
#include <string>

struct Robot {
    virtual ~Robot() = default;
    virtual std::string kind() const { return "ROBOT"; }
};
struct Sentry : Robot {
    std::string kind() const override { return "SENTRY"; }
};

int main() {
    Sentry object;
    const Robot &ref = object;
    Robot sliced = object;
    std::cout << ref.kind() << ' ' << sliced.kind() << '\n';
    return 0;
}
```

执行推导

引用保留动态类型，输出 SENTRY；基类值已经发生切片，输出 ROBOT。

常见错误及原因

不要用 vector<Base> 保存希望保留动态行为的不同派生对象。

自检

override 能否发现派生方法遗漏 const 的错误?

参考解释： 可以，该签名不匹配会导致编译错误。

9.3 拓展：值复制、移动与资源所有权

原 Band 只含 string，默认复制会复制成员值，通常不需要手写析构、复制构造与赋值。含 vector 或 string 的普通数据对象优先依赖这些标准库成员管理资源，这称为 Rule of Zero。不要为了学习面向对象而先给每个类添加 new 和 delete。

unique_ptr 表达独占所有权，不能复制，但可以移动。移动 unique_ptr 后源保证为空，目标接管资源；不能把这个明确保证推广为“所有移动后的对象都为空”。普通 string 或 vector 移动后仍有效，但不应依赖未由接口保证的具体值。

RAII 把资源释放与对象生命周期绑定：正常离开作用域或异常展开时，已完成构造的自动对象会析构。挑战题中的深复制缓冲区和 clone 是额外的所有权练习，需要在理解原 Band 值语义之后再做，不作为对原课程顺序的替代。

完整课堂实验

L2_C09_D03.cpp · CPP-Classroom source

```cpp
#include <iostream>
#include <memory>
#include <utility>

int main() {
    std::unique_ptr<int> first = std::make_unique<int>(7);
    std::unique_ptr<int> second = std::move(first);
    std::cout << std::boolalpha << (first == nullptr) << ' ' << *second << '\n';
    return 0;
}
```

执行推导

输出 true 7，只有 second 拥有该整数。作用域结束时资源自动释放，不需要裸 delete。

常见错误及原因

移动后不能继续无条件解引用 first；它为空。

自检

Band 内部都是 string，是否为了“完整”就必须手写深复制函数？

参考解释： 不需要，默认成员复制已经提供所需值语义；自行管理裸资源时才需要重新审视特殊成员函数。

本章配套例题

ID	题目
O21	组合两个电机
O22	电池与动作组合
O23	注入可替换配置
O24	只读状态快照
O25	诊断消息集合
O31	virtual 动态派发
O32	纯虚接口
O33	对象切片
O34	虚析构函数
O35	多态容器
O36	独占所有权转移
O37	Rule of Zero
O38	异常与析构清理
O39	不可复制的设备
O40	声明与类外定义